

Teoría de Backtracking - Ejercicios Conceptuales (01-05)

Este documento contiene las respuestas a los ejercicios teóricos de la guía de Backtracking de la Escuela de Computación (UCV).

1. Definición de "Búsqueda Exhaustiva" en Backtracking

La **búsqueda exhaustiva** es una estrategia de resolución de problemas que explora todas las combinaciones posibles de una solución potencial para encontrar una (o todas) las respuestas correctas.

En el contexto de **Backtracking**, es una búsqueda exhaustiva "inteligente". A diferencia de la fuerza bruta pura, el backtracking construye la solución pieza por pieza y utiliza **podas** (restricciones) para abandonar una rama del árbol de búsqueda en cuanto determina que no puede conducir a una solución válida.

Ejemplo: En un Sudoku, la búsqueda exhaustiva probaría números del 1 al 9. Si el backtracking ve que poner un 5 rompe la regla de la fila, deja de explorar esa rama y "vuelve atrás" inmediatamente.

2. Estados y Variables en la Llamada Recursiva

Cada llamada recursiva en un algoritmo de backtracking crea un nuevo **marco de activación** (estado) en la pila de ejecución. Este estado se define por:

1. **Variables Locales (Parámetros):** Representan el progreso actual (ej. `nivel` de recursión, `sumaAcumulada`, `posicionActual`).
2. **Estado Global/Compartido:** Generalmente un arreglo o matriz que representa la estructura del problema (ej. `Tablero`, `Visitados`).
3. **Ámbito de Decisión:** El bucle (usualmente un `for`) que define las opciones disponibles para ese nivel específico.

Importancia: Al retornar de la recursión, es vital restaurar el estado original (desmarcar variables globales) para que otras ramas del árbol no se vean afectadas por decisiones fallidas previas.

3. Principales Esquemas de Solución en Backtracking

Existen tres esquemas fundamentales dependiendo del objetivo del problema:

- **Encontrar UNA solución:** El algoritmo retorna un booleano. Tan pronto como halla una respuesta válida, detiene la recursión.
 - **Encontrar TODAS las soluciones:** El algoritmo explora el árbol completo (con podas) y guarda o imprime cada solución encontrada en el caso base.
 - **Encontrar la MEJOR solución (Optimización):** Se lleva una variable global (ej. `MIN_COSTO`) y se actualiza cada vez que se llega a una solución mejor que la anterior.
-

4. Determinación de Existencia y Conjunto de Soluciones

¿Cómo saber si tiene solución?

Se utiliza una función recursiva de tipo `bool`.

- **Caso Base:** Si se cumplen las condiciones de éxito, retorna `true`.
- **Paso Recursivo:** Si alguna llamada hija retorna `true`, la función actual propaga ese `true` hacia arriba. Si se agotan las opciones sin éxito, retorna `false`.

¿Cómo determinar todo el conjunto de soluciones?

Se utiliza una función de tipo `void`.

- **Caso Base:** En lugar de retornar, se imprime la solución actual o se añade a una lista global.
- **Paso Recursivo:** Se exploran todas las ramas válidas sin detenerse tras el primer éxito.

5. La Mejor Solución en Backtracking

Para encontrar la **mejor solución** (optimización), el algoritmo debe:

1. **Explorar exhaustivamente:** No detenerse ante la primera solución hallada.
2. **Variable de Control:** Mantener una variable global (ej. `MejorCosto`) inicializada en un valor extremo (infinito para mínimos, cero para máximos).
3. **Actualización:** Cada vez que se alcanza un caso base válido, se compara el costo de la solución actual con `MejorCosto`. Si es mejor, se actualiza la variable y se guarda la configuración (ej. en un arreglo `MejorCamino`).
4. **Poda por Cota:** Si en cualquier punto del camino el costo acumulado ya es peor que el `MejorCosto` conocido, se aborta esa rama (recursión) inmediatamente.

Ejemplo: En el problema del Laberinto, si ya encontramos un camino de 10 casillas, y el camino actual que estamos explorando ya lleva 11, dejamos de buscar por ahí porque nunca será "el mejor".

Ejemplo Práctico: El problema de las N-Cifras (Ejercicio 7)

Para ilustrar estos conceptos, usemos el ejercicio de los números divisibles:

- **Búsqueda Exhaustiva:** Probar dígitos del 1 al 9.
- **Estado:** `NumeroActual` y `n_CantActual` (posición).
- **Poda:** `NuevoNumero % n_CantActual == 0`.
- **Backtracking (Estado):** Al terminar de probar un número, se desmarca en el arreglo `NumerosUsados` para dejar el camino limpio a la siguiente opción del bucle.